

Universidad Cenfotec
Proyecto de Ingenieria de Software 3

Estructura del Proyecto Brakket

Guia de organizacion de carpetas y archivos

Equipo Coffee&Commits
II Cuatrimestre 2026

Tabla de Contenidos

1. Vision General de la Arquitectura

Brakket se estructura bajo una arquitectura en capas, en la que cada capa tiene una responsabilidad bien definida y se comunica unicamente con las capas adyacentes. Esto facilita el mantenimiento, permite el trabajo en paralelo del equipo y mantiene una separacion clara entre la interfaz, la logica de negocio y los datos.

- Capa de presentacion (Front-end): desarrollada en Angular. Renderiza las vistas, gestiona la navegacion y construye una interfaz responsiva. No contiene logica de negocio; se comunica con el back-end mediante una API REST.
- Capa de aplicacion (Back-end): desarrollada en Java con Spring Boot y gestionada con Maven. Se organiza en tres niveles: controladores (exponen la API REST), servicios (logica de negocio) y repositorios (acceso a datos). Tambien se comunica con los servicios externos.
- Capa de persistencia: base de datos relacional PostgreSQL, accedida por el back-end a traves de los repositorios.
- Servicios externos: integracion con Twitch y el analisis de audiencia con IA.

El repositorio se organiza como un monorepo con dos aplicaciones independientes (backend y frontend) que espejan los mismos modulos de dominio, de modo que el equipo pueda trabajar en paralelo sobre cada modulo del ERS.

Nota: Convenciones de nomenclatura del equipo: clases Java en *PascalCase*, archivos Angular en *kebab-case* y base de datos en *snake_case*.

2. Raiz del Repositorio (monorepo)

La raiz agrupa las dos aplicaciones, la documentacion academica, la configuracion de integracion continua y los archivos de orquestacion.

```
brakket/
|-- backend/                # API Java Spring Boot
|-- frontend/              # Aplicacion Angular
|-- docs/                  # Documentacion academica
|   |-- acta-constitucion.docx
|   |-- plan-general.docx
|   |-- ers.docx
|   `-- informe-diseno.docx
|-- .github/
|   `-- workflows/
|       |-- backend-ci.yml  # Build + tests JUnit
|       `-- frontend-ci.yml # Build + tests Karma
|-- docker-compose.yml     # PostgreSQL + servicios
|-- .gitignore
`-- README.md
```

3. Backend — Java Spring Boot

El back-end se construye con Maven. El código fuente vive bajo el paquete base `com.coffeecommits.brakket` y se organiza por módulo de dominio; cada módulo contiene internamente sus tres niveles (controller, service, repository) más sus entidades (model) y objetos de transferencia (dto).

3.1. Estructura raíz y recursos

```
backend/
|-- pom.xml                # Dependencias Maven
`-- src/
    |-- main/
    |   |-- java/com/coffeecommits/brakket/
    |   |-- resources/
    |       |-- application.yml
    |       |-- application-dev.yml
    |       |-- application-prod.yml
    |       |-- db/migration/    # Flyway
    |           |-- V1__init_schema.sql
    |           |-- V2__users_roles.sql
    |           |-- V3__tournaments.sql
    |-- test/
    |   |-- java/com/coffeecommits/brakket/
```

Nota: Las migraciones de base de datos se versionan con Flyway en `resources/db/migration`, garantizando que el esquema PostgreSQL sea reproducible.

3.2. Paquete base: config y common

```
com/coffeecommits/brakket/
|-- BrakketApplication.java    # Punto de entrada
|-- config/
|   |-- SecurityConfig.java
|   |-- CorsConfig.java
|   |-- SwaggerConfig.java
|   |-- JwtConfig.java
|-- common/
|   |-- exception/
|       |-- GlobalExceptionHandler.java
|       |-- ResourceNotFoundException.java
|       |-- BusinessException.java
|   |-- dto/
|       |-- ApiResponse.java
|   |-- util/
|       |-- DateUtils.java
```

3.3. Módulo de dominio — patrón estándar (auth)

Todos los módulos siguen exactamente esta misma estructura interna de cinco carpetas:

```
auth/                                # Módulo (patrón para TODOS)
|-- controller/
|   |-- AuthController.java          # Expone API REST
|-- service/
|   |-- AuthService.java              # Interfaz
|   |-- AuthServiceImpl.java          # Lógica de negocio
|-- repository/
|   |-- UserRepository.java           # Acceso a datos (JPA)
|-- model/
|   |-- User.java                     # Entidad (PascalCase)
```

```
|  `-- Role.java
|-- dto/
|   |-- LoginRequest.java
|   |-- RegisterRequest.java
|   `-- AuthResponse.java
```

3.4. Ejemplo de modulo mayor (tournament)

```
tournament/                                     # Torneos + fixtures/brackets
|-- controller/
|   |-- TournamentController.java
|   `-- FixtureController.java
|-- service/
|   |-- TournamentService.java
|   |-- TournamentServiceImpl.java
|   `-- BracketGeneratorService.java
|-- repository/
|   |-- TournamentRepository.java
|   `-- MatchRepository.java
|-- model/
|   |-- Tournament.java
|   |-- Match.java
|   `-- Fixture.java
|-- dto/
|   |-- TournamentRequest.java
|   `-- TournamentResponse.java
```

Los modulos restantes replican el mismo patron controller / service / repository / model / dto:

- team/ — gestion de equipos y transferencias (Team, Transfer).
- dispute/ — resolucion de disputas (Dispute).
- twitch/ — integracion con Twitch (TwitchIntegrationService, cliente de API externa).
- analytics/ — analisis de sentimiento con IA (SentimentAnalysisService).
- sponsorship/ — patrocinios (Sponsorship, SponsorSlot).
- notification/ — notificaciones del sistema.
- statistics/ — estadisticas y metricas.
- progression/ — progresion y personalizacion del jugador.
- admin/ — panel de administracion de la plataforma.

3.5. Pruebas (JUnit 5 + Mockito)

La carpeta de pruebas espeja la estructura de main: cada servicio y controlador tiene su clase de prueba correspondiente.

```
test/java/com/coffeecommits/brakket/
|-- auth/
|   |-- AuthServiceTest.java      # JUnit 5 + Mockito
|   `-- AuthControllerTest.java
|-- tournament/
|   |-- TournamentServiceTest.java
|   `-- BracketGeneratorServiceTest.java
|-- ...                          # Un test por service/controller
```

4. Frontend — Angular

El front-end separa el código en cuatro grandes bloques: core (servicios de una sola instancia, guards e interceptors), shared (elementos reutilizables), layout (estructura visual) y features (un módulo por dominio, cargado de forma lazy). No contiene lógica de negocio: valida entradas y consume la API REST.

4.1. Estructura raíz y configuración

```
frontend/
|-- package.json
|-- angular.json
|-- tsconfig.json
|-- karma.conf.js          # Config pruebas Jasmine/Karma
`-- src/
    |-- main.ts
    |-- index.html
    |-- styles.scss        # Estilos globales (tema oscuro)
    |-- environments/
    |   |-- environment.ts
    |   `-- environment.prod.ts
    |-- assets/
    |   |-- images/
    |   `-- styles/
    |       |-- _variables.scss # #2563EB, #22D3EE
    |       `-- _theme.scss
    `-- app/
```

4.2. Carpeta app (core, shared, layout, models)

```
app/
|-- app.component.ts
|-- app.routes.ts
|-- app.config.ts
|-- core/                  # Singletons (una sola instancia)
|   |-- guards/
|   |   |-- auth.guard.ts
|   |   `-- role.guard.ts
|   |-- interceptors/
|   |   |-- jwt.interceptor.ts
|   |   `-- error.interceptor.ts
|   `-- services/
|       |-- auth.service.ts
|       `-- token.service.ts
|-- shared/                # Reutilizable
|   |-- components/
|   |   |-- button/
|   |   |-- modal/
|   |   `-- table/
|   |-- pipes/
|   `-- directives/
|-- layout/
|   |-- header/
|   |-- sidebar/
|   `-- footer/
|-- models/
|   |-- user.model.ts
|   |-- tournament.model.ts
|   `-- team.model.ts
`-- features/
```

4.3. Feature — patron estandar (tournaments)

Cada feature agrupa sus rutas, paginas, componentes y servicios. Cada componente esta formado por cuatro archivos: .ts (logica), .html (plantilla), .scss (estilos) y .spec.ts (prueba con Jasmine/Karma).

```
features/tournaments/          # Feature (patron para TODOS)
|-- tournaments.routes.ts      # Rutas lazy-loaded
|-- pages/
|   |-- tournament-list/
|   |   |-- tournament-list.component.ts
|   |   |-- tournament-list.component.html
|   |   |-- tournament-list.component.scss
|   |   `-- tournament-list.component.spec.ts
|   `-- tournament-detail/
|       `-- (mismos 4 archivos)
|-- components/
|   `-- bracket-view/
|       `-- (mismos 4 archivos)
`-- services/
    `-- tournament.service.ts
```

Los demas features replican el mismo patron pages / components / services:

- auth/ — inicio de sesion, registro y roles.
- teams/ — equipos y transferencias.
- disputes/ — gestion de disputas.
- twitch/ — transmision e integracion con Twitch.
- analytics/ — tableros de analisis de audiencia.
- sponsorships/ — espacios y gestion de patrocinios.
- notifications/, statistics/, progression/, admin/ — modulos restantes.

5. Convenciones y Reglas Clave

- Separacion estricta: el front-end no contiene logica de negocio; toda regla vive en los servicios del back-end.
- Modulos espejados: los mismos modulos del ERS existen en back-end y front-end, permitiendo el trabajo en paralelo del equipo.
- Nomenclatura: clases Java en PascalCase, archivos Angular en kebab-case, base de datos en snake_case.
- Pruebas obligatorias: cada service del back-end lleva su prueba JUnit 5; cada componente del front-end lleva su archivo .spec.ts.
- Colores de marca: azul #2563EB y cian neon #22D3EE sobre tema oscuro, definidos en _variables.scss.
- Integracion continua: GitHub Actions ejecuta build y pruebas de ambas aplicaciones en cada push (backend-ci.yml y frontend-ci.yml).